



Programmation Scripts Intégrés

SED-1515 A00

Projet Groupe - II

Conception Fonctionnelle Initiale Détaillée

Groupe 3

Kelly-Ann Lessard 300307532

David Berosy 300511667

Ange Yvan Mugisha 300505715

Artem Lavrynets 300429506

David Mufuta 300383643

Présenté à : Yassine Hammadi

TA : AudreyAnn Bleau-Brousseau

Université d'Ottawa

Date de soumission : 9 novembre 2025

Table des Matières

Introduction.....	4
1. Objectif du Programme.....	5
2. Listes des Modules Utilisées.....	6
2.1 VS Code.....	6
2.2 Outils.....	6
2.2.1 Git & GitHub.....	6
2.2.2 Copilot.....	6
3. Types de Données Utilisés.....	7
3.1. Modules à écrire.....	7
3.2. Modules Python utilisés.....	8
4. Code Mise à Jour.....	9
Tableau #1: Code Mise à Jour.....	9
Tableau #2: Câblage.....	10
5. Tests.....	11
6. Calendrier Mise à Jour.....	12
Photo #2: Deuxième Tâche de Projet Mise à Jour.....	13
Référence.....	15

Introduction

Le présent document constitue le livrable B du projet en groupe. Il s'agit de la conception fonctionnelle initiale détaillée du projet de brachiographe, un dispositif électromécanique permettant de tracer des formes sur papier à l'aide d'un bras articulé contrôlé par un microcontrôleur Raspberry Pi Pico.

Ce livrable vise à présenter l'organisation fonctionnelle complète du code, la structure d'appel des fonctions, ainsi que la définition des entrées et sorties de chaque module logiciel.

Le programme est conçu pour lire deux entrées analogiques (potentiomètres) et une entrée numérique (interrupteur), afin de manipuler un bras robotisé composé de trois servomoteurs simulant une articulation d'épaule, de coude et de stylo. L'objectif est de contrôler en temps réel le mouvement du bras de dessin de manière fluide et précise, à la manière d'un jouet Etch-A-Sketch.

1. Objectif du Programme

L'objectif principal de cette étape du projet est de développer le squelette fonctionnel complet du logiciel de commande du brachiographe.

Cette version du code ne contient pas encore les algorithmes complets, mais elle définit :

- l'ensemble des fonctions nécessaires au fonctionnement du système, avec leurs signatures (entrées, sorties et responsabilités) ;
- la structure d'appel entre les différents modules (`main.py`, `adc_reader.py`, `inverse_kinematics.py`, `pwm_controller.py`, `pen_control.py` et `utils.py`) ;
- la répartition des tâches entre les membres du binôme, selon leurs responsabilités dans le développement de chaque fonction ;
- les structures de données prévues pour assurer la communication entre les modules ;
- L'utilisation de GitHub pour la gestion du code source, la documentation des bogues et le suivi de l'avancement du développement.

Ce livrable permet donc d'assurer que la conception logicielle est modulaire, documentée et cohérente avant le début de l'implémentation réelle du code et des tests unitaires.

Le programme final, basé sur cette conception, permettra au Raspberry Pi Pico de lire les signaux d'entrée, de calculer la position du bras via la cinématique inverse, et de piloter les servomoteurs grâce à des signaux PWM précis, assurant un mouvement stable et contrôlé du traceur.

2. Listes des Modules Utilisées

2.1 VS Code

L'environnement de développement principal demeure Visual Studio Code, utilisé pour la création et la structuration du code squelettique.

Les extensions suivantes ont été exploitées :

- Pico-Go : pour flasher et interagir avec le Raspberry Pi Pico ;
- Python : pour la coloration, l'autocomplétion et le linting ;
- GitHub Pull Requests & Issues : pour la gestion du code et le suivi des tâches.

VS Code offre une interface intégrée et flexible, facilitant la gestion du projet, la rédaction des fonctions et la synchronisation avec GitHub.

2.2 Outils

2.2.1 Git & GitHub

GitHub est utilisé pour :

- héberger le code squelettique sur une branche dédiée ;
- suivre les bogues et tâches via les *issues* ;
- attribuer les responsabilités à chaque membre du binôme ;
- assurer la traçabilité du développement grâce au contrôle de version.

Chaque fonction définie dans le squelette possède une *issue* correspondante, permettant un suivi clair et collaboratif de l'avancement.

2.2.2 Copilot

GitHub Copilot va être utilisé comme outil d'assistance intelligente pour :

- rédiger les en-têtes et commentaires des fonctions ;
- standardiser les définitions (entrées, sorties, descriptions) ;
- améliorer la clarté et la cohérence du code squelettique.

Copilot soutiendra la rédaction, la planification et la documentation du projet en garantissant rigueur et efficacité dans la production des livrables.

3. Types de Données Utilisés

3.1. Modules à écrire

- `adc_reader.py` :
Ce module assure la lecture des valeurs analogiques provenant des deux potentiomètres X et Y à l'aide des entrées ADC du Raspberry Pi Pico. Il inclut également un filtrage simple pour réduire le bruit et améliorer la stabilité des mesures.
- `inverse_kinematics.py` :
Il calcule les angles nécessaires pour les servomoteurs de l'épaule et du coude à partir des coordonnées X et Y lues. Cette transformation inverse cinématique permet de positionner le bras du brachiographe avec précision.
- `pen_control.py` :
Ce module gère le mouvement du stylo. Il lit l'état de l'interrupteur et commande le servo correspondant pour lever ou abaisser le stylo lors du dessin.
- `pwm_controller.py` :
Il génère les signaux PWM envoyés aux servomoteurs. Chaque servo reçoit un signal correspondant à l'angle calculé. Ce module centralise la commande et la synchronisation

des moteurs.

- `utils.py` :

Contient des fonctions utilitaires comme la conversion de plages de valeurs, la limitation d'angles et le filtrage de données. Il regroupe les fonctions réutilisables dans plusieurs parties du programme.

- `main.py` :

Point d'entrée du programme. Ce fichier orchestre l'ensemble des modules : il lit les entrées, calcule les angles nécessaires, envoie les signaux PWM et gère la position du stylo.

3.2. Modules Python utilisés

- `machine` : utilisé pour accéder aux classes `Pin`, `ADC` et `PWM` permettant la communication directe avec le matériel du Raspberry Pi Pico.
- `time` / `utime` : utilisé pour gérer les délais, temporisations et pauses nécessaires à la synchronisation des servomoteurs.
- `math` : utilisé pour les calculs trigonométriques essentiels à la transformation inverse cinématique du bras.

4. Code Mise à Jour

Tableau #1: Code Mise à Jour

```
brachiographe
├── main.py                # Point d'entrée du programme - boucle principale
├── adc_reader.py          # Lecture des potentiomètres X et Y + lecture de l'interrupteur
│   ├── read_potentiometers() # Retourne les valeurs X et Y lues par les ADC
│   └── read_switch()         # Retourne l'état (haut/bas) du stylo via un interrupteur
├── inverse_kinematics.py  # Calcul des angles d'épaule et de coude |
│   └── calculate_angles(x, y) # Retourne les angles  $\alpha$  et  $\beta$  en degrés
├── pwm_controller.py      # Commande les servos via signaux PWM
│   ├── move_servos(alpha, beta) # Déplace les servos selon les angles calculés
│   └── setup_pwm(pin)           # Initialise un signal PWM sur une broche donnée
├── pen_control.py         # Gestion du stylo (haut/bas)
│   └── set_pen_state(state)     # Active ou désactive le servo du stylo
└── utils.py              # Fonctions utilitaires (filtrage, conversion, limites)
    ├── map_value(value, in_min, in_max, out_min, out_max) # Conversion de plages
    └── limit_angle(angle, min_angle, max_angle)           # Limite de sécurité
```

Tableau #2: Câblage

```
from machine import Pin, ADC, PWM

# -- Entrées analogiques --
# Potentiomètre X branché sur GP26 (ADC0)
pot_x = ADC(26)

# Potentiomètre Y branché sur GP27 (ADC1)
pot_y = ADC(27)

# Entrée numérique
# Interrupteur du stylo sur GP16 (avec résistance de tirage interne)
switch = Pin(16, Pin.IN, Pin.PULL_UP)

# Sorties PWM pour les servos
# Servo de l'épaule sur GP15
servo_epaule = PWM(Pin(15))

# Servo du coude sur GP14
servo_coude = PWM(Pin(14))

# Servo du stylo sur GP13
servo_stylo = PWM(Pin(13))

# -- Paramètres communs --
# Fréquence PWM à 50 Hz pour tous les servos
for s in [servo_epaule, servo_coude, servo_stylo]:
    s.freq(50)
```

5. Tests

Pour valider le fonctionnement du brachiographe, plusieurs tests fonctionnels ont été définis :

- Lecture des potentiomètres : le module `adc_reader.py` sera testé en lisant les tensions analogiques X et Y. Le résultat attendu est l'obtention de valeurs numériques stables comprises entre 0 et 3.3 V. Ce test est simple et devrait prendre environ 30 minutes.
- Filtrage des valeurs analogiques : le module `utils.py` sera évalué en utilisant une série de lectures fluctuantes. Le système doit produire une valeur lissée ou une moyenne glissante. Ce test est de difficulté modérée et nécessitera environ 45 minutes.
- Transformation inverse cinématique : le module `inverse_kinematics.py` recevra des coordonnées X/Y et devra calculer des angles articulaires réalistes pour l'épaule et le coude. Ce test est complexe car il implique des calculs trigonométriques, et il demandera environ 1 h 30. Conversion angle vers PWM : le module `pwm_controller.py` sera testé en convertissant un angle en degrés en un rapport cyclique PWM adapté à chaque servomoteur. Ce test est de difficulté modérée et prendra environ 1 heure.
- Commande du stylo haut/bas : le module `pen_control.py` sera vérifié en utilisant un interrupteur ON/OFF. Le résultat attendu est que le servo du stylo s'active ou se désactive correctement selon l'état de l'interrupteur. Ce test est simple et devrait durer environ 30 minutes.
- Test d'intégration complet : l'ensemble du système (`main.py` et tous les modules) sera testé en manipulant les potentiomètres et l'interrupteur. Le bras doit se déplacer de manière fluide et produire un traçage cohérent. Ce test est complexe et nécessitera environ 2 heures.
- Test de sécurité mécanique : la fonction de protection fournie sera testée avec des valeurs extrêmes de X et Y. Le bras doit limiter ses mouvements afin d'éviter des angles dangereux. Ce test est de difficulté modérée et prendra environ 1 heure.

6. Calendrier Mise à Jour

Pour ce projet nous avons utilisé l'application Notion pour le calendrier de projet et le dressage et répartition des tâches. Voici le [lien Notion](#). [2]

SED1515 Projet Groupe 3

Translate to English Share

Tâches de projet

Table Gantt Calendar

Status Nom de tâche Date d'échéance Jalons Dependait sur Priorité

✓	Déterminer l'Objectif	October 30, 2025 23:59	Tâche		Moyen
✓	Calendrier Notion	October 31, 2025 23:59	Tâche		Moyen
✓	Créer un Plan de Conception	October 31, 2025 23:59	Tâche	Déterminer l'Objectif	Moyen
✓	Livrable A	November 2, 2025 23:59	Livrable	Calendrier Notion Créer un Plan de Conception Déterminer l'Objectif	Haut
✓	GitHub Partage Repository	November 4, 2025	Tâche		Haut
✓	Script Python Conception Fonctionnelle	November 7, 2025	Tâche	GitHub Partage Repository	Moyen
✓	Rencontre d'Équipe	November 7, 2025 13:00	Rencontre		
□	Créations des Tests	November 9, 2025	Tâche		Moyen
□	Livrable B	November 9, 2025 0:00	Livrable	Livrable A GitHub Partage Repository Script Python Conception Fonctionnelle	Haut

Photo #1: Tâches de Projet Mise à Jour

Référence

[1] N. Panchal, “Raspberry Pi Pico Servo Motor Control - Electric DIY Lab,” *Electric DIY Lab*, Apr. 02, 2021.

https://electricdiylab.com/raspberry-pi-pico-servo-motor-control/?utm_source=chatgpt.com
(accessed Nov. 02, 2025).

[2] “The AI workspace that works for you. | Notion,” *Notion*, 2025.

https://www.notion.so/SED1515-Projet-Groupe-3-26eb170387ad81a1b932f72c58271b27?source=copy_link (accessed Nov. 02, 2025).

[3] Microsoft Copilot – <https://copilot.microsoft.com>